

Intermediate Code Generation

05 May, 2026

Three-address codes

It is a sequence of statements of the general form

$$x = y \text{ op } z$$

where x , y and z are names, constants, or compiler generated temporaries and op stands for any operator.

Example: A source language expression like

$$x + y * z$$

might be translated into a sequence

$$\begin{aligned} t_1 &= y * z \\ t_2 &= x + t_1 \end{aligned}$$

where t_1 and t_2 are temporaries.

Consider the assignment statement.

$$a = b * -c + b * -c$$

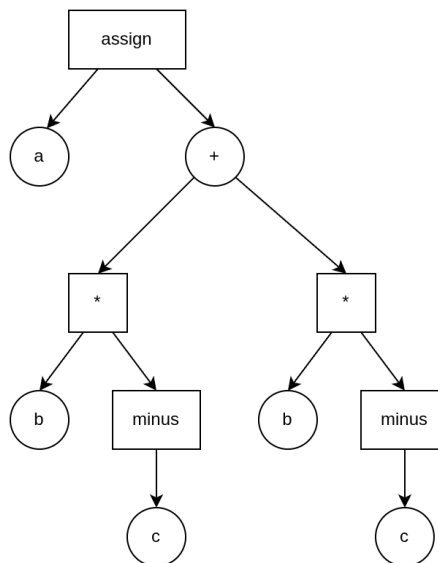


Figure 1: Syntax tree

Three address code:

$$\begin{aligned} t_1 &= \text{minus } c & t_2 &= b * t_1 \\ t_3 &= \text{minus } c & t_4 &= b * t_3 \\ t_5 &= t_2 + t_4 & a &= t_5 \end{aligned}$$

Some common three-address codes

1. Assignment statements of the form

$$x = y \text{ op } z$$

where op is a binary operator.

2. Assignment instruction of the form

$$x = \text{op } y$$

where op is a unary operator.

3. Copy statements of the form

$$x = y$$

where the value of y is copied to x .

4. The unconditional jump

$$\text{goto } L$$

The three-address statement with label L is the next to be executed.

5. Conditional jumps such as

$$\text{if } x \text{ relop } y \text{ goto } L$$

6. Indexed assignments of the form

$$x = y[i]$$

This statement sets x to the value in the location i units beyond location y .

$$x[i] = y$$

This statement sets the location i units beyond x to y .

7. Address and pointer assignments of the form

$$x = \&y$$

This statement stores the location of y in x . (There is a distinction between the meaning of identifiers on the left and right sides of an assignment. In each of the assignments, $i = 5$ and $i = i + 1$, the right side specifies an integer value while the left side specifies where the value is to be stored. The term *r-value* are what we usually think as values while *l-values* are locations.)

$$x = *y$$

In this statement, presumably y is a pointer or a temporary whose *r-value* is a location. The *r-value* of x is made equal to the contents of the location.

$$*x = y$$

This statement sets the *r-value* of the object pointed to by x to the *r-value* of y .

Implementation of three-address codes

Quadruples

A *quadruple* is a record structure with four fields which are called op, arg₁, arg₂ and result.

The three address statement,

$$x = y \text{ op } z$$

is represented by replacing y in arg₁, z in arg₂ and x in result.

- # Statements with unary expressions like $x = -y$ or $x = y$ do not use arg_2 .
- # The quadruples for the assignment

$$a = b * -c + b * -c$$

are:

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

Figure 2: Quadruple structure

Triples

- # Here three-address statements can be represented by records with only three fields op , arg_1 and arg_2 .
- # The fields arg_1 and arg_2 for the argument of op are either pointers to the symbol table (for program defined names or constants) or pointers into the triple structure (for temporary values).
- # Parenthesized numbers represent pointers into the triple structures while symbol table pointers are represented by the names itself.
- #

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

Figure 3: Triple structure

Indirect triples

- # Another implementation of three-address code that has been considered is that of listing pointers to triples rather than listing the triples themselves.
- # Example: Let us use an array *statements* to list pointers to triples in the desired order.

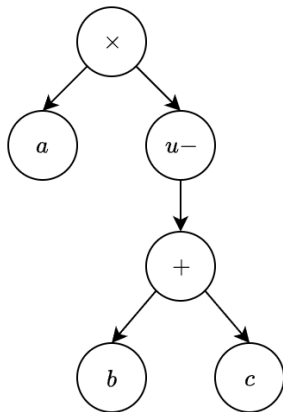
Problem

Translate the following expression:

$$a * -(b + c)$$

into a syntax tree, three-address code and quadruple.

Syntax tree



,

Three-address code

$$t_1 = b + c$$

$$t_2 = \text{minus } t_1$$

$$t_3 = a * t_2$$

Quadruple

op	arg ₁	arg ₂	result
+	<i>b</i>	<i>c</i>	<i>t</i> ₁
minus	<i>t</i> ₁		<i>t</i> ₂
×	<i>a</i>	<i>t</i> ₂	<i>t</i> ₃